PART II — BUILDING THE INGESTION PIPELINE

Chapter 7

Chunking Strategies

"The unit you retrieve is the unit you reason with. Choose it badly and no amount of clever retrieval will save you."

Chapter 7 — Chunking Strategies

If the Document object is the atom of the ingestion pipeline, the chunk is the ion — a Document that has been broken apart to carry charge. Every retrieval system ultimately retrieves chunks, ranks chunks, feeds chunks to the LLM, and cites chunks back to the user. The decisions you make about how to produce those chunks — where to cut, how long to make them, how much to let them overlap — propagate silently through every downstream component. A poor chunking strategy cannot be rescued by a better embedding model or a more sophisticated retriever.

This chapter walks through the full spectrum of chunking approaches available in a LangChain-based RAG system, from the simplest fixed-size cut to semantically driven, structure-aware splitting. Along the way you will encounter the defaults used in the pipeline built in this book (chunk_size=1000, chunk_overlap=200) and, more importantly, the reasoning behind those defaults and the conditions under which you should deviate from them. The chapter closes with a practical experiment showing how chunking choices produce measurable, visible differences in retrieval quality.

By the end of this chapter you will be able to choose an appropriate splitting strategy for any document type, configure the key parameters with confidence, and audit your chunks for common quality problems before they reach the vector store.

7.1 Why Chunk at All? Context Windows and Retrieval Precision

The answer to "why chunk?" has two separate parts, and both are necessary. Treating them as one leads to confusion about what chunking is actually optimizing for.

The context window constraint

Every LLM accepts a finite number of tokens as input — the context window. For the llama-3.1-8b-instant model used in this pipeline, that window is 128,000 tokens; for GPT-4o it is 128,000; for smaller models it may be 4,096 or 8,192. A single long document — a 200-page legal contract, a technical manual, a research paper — easily contains 50,000 to 200,000 tokens. You cannot feed the entire document to the model.

Even if you could, you would not want to. The "lost in the middle" problem (Chapter 24) shows that transformer attention dilutes as context grows: facts buried in the middle of a 100,000-token context are attended to far less reliably than facts near the start or end. Precision degrades before you hit the hard limit. The practical upper bound for trustworthy, instruction-following attention on an 8B model is roughly 16,000 tokens — far below the headline figure.

The retrieval precision problem

The second reason to chunk is retrieval precision. The retriever works by comparing a query embedding against document embeddings. If your "document" is a 5,000-word chapter covering ten topics, its embedding is a centroid of all those topics — a blurry average that lands nowhere in particular in vector space. A query about one specific fact may not rank that document highly, even though the answer is inside it.

Smaller chunks produce more focused embeddings. A 250-token chunk that covers exactly one concept has an embedding that is a sharp, high-confidence point in semantic space, not a diffuse cloud. Retrieval matches

query to concept, not query to collection-of-concepts. This is the deeper reason why chunking matters: it is not just about fitting text into context windows, it is about making the vector index semantically precise.

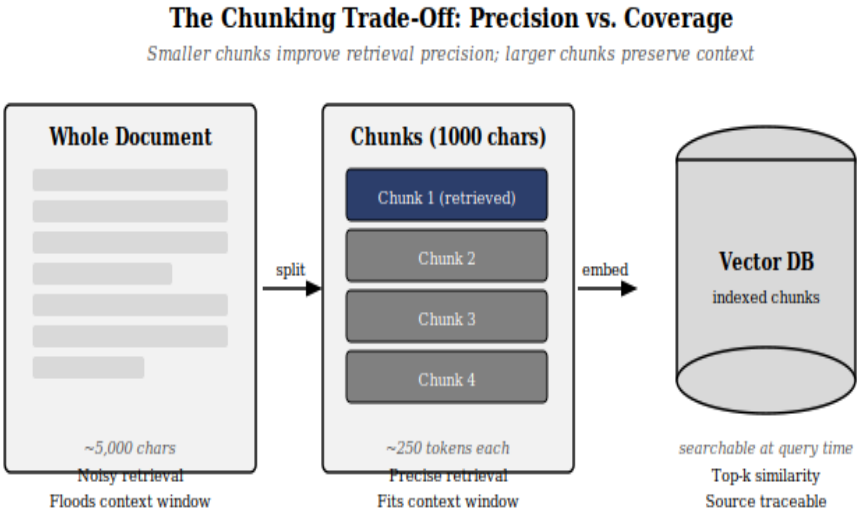

Figure 7.1 — Why chunking works: smaller units produce sharper embeddings, enabling precise similarity search. Without chunking, a single large document embedding dilutes all its topics into one blurry centroid.

Definition — Chunk

A sub-document produced by splitting a LangChain Document object at ingestion time. A chunk inherits the parent Document's metadata wholesale and adds two splitter-time fields: chunk_index (its ordinal position within the parent) and total_chunks (how many siblings it has). The chunk, not the original document, is the unit that gets embedded, stored, retrieved, and fed to the LLM.

Analogy — The index card system

A library card catalogue does not give you a shelf — it gives you a card. Each card describes one narrow topic from one book. A query for "Byzantine mosaics" returns the card for that topic, not the entire art-history section. Chunking your documents is the act of writing those cards. The quality of your retrieval system is, at root, the quality of your card-writing.

7.2 Fixed-Size Chunking

Fixed-size chunking is the simplest possible strategy: divide the document into segments of exactly N characters, with an optional overlap of M characters between adjacent segments. It requires no understanding of the text's structure, no separator detection, and no linguistic analysis.

```
from langchain_text_splitters import CharacterTextSplitter

splitter = CharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200,
    separator='',        # no separator — pure char count
    length_function=len,
)

chunks = splitter.split_documents(documents)
```

The appeal is obvious: predictable chunk sizes, trivial to implement, no dependencies. The problem is equally obvious: the splitter cuts at arbitrary byte positions with no regard for sentence or paragraph boundaries. A chunk can end mid-sentence, mid-word, even mid-token for multi-byte Unicode characters. The resulting chunks are semantically incoherent at their edges, which degrades both the embedding quality and the readability of retrieved passages.

Fixed-size chunking has one legitimate use case: documents with no natural language structure — binary exports, log files, purely numeric data — where there is no meaningful sentence to preserve. For prose, code, or any human-written text, the recursive splitter described in Section 7.3 is strictly superior.

> **Common pitfall — Treating chunk_size as a token count**
>
> CharacterTextSplitter and RecursiveCharacterTextSplitter measure size in characters by default, not tokens. A chunk_size=1000 setting produces chunks of at most 1000 characters — which is approximately 250 tokens in English prose, but can be 350–600 tokens for code, URLs, or numeric data. If your model has a hard token limit per input, measure in tokens using a token-counting length_function (see Section 7.5 for the tiktoken-based approach). Confusing characters for tokens is one of the most common causes of silent context-window overflow in production pipelines.

7.3 RecursiveCharacterTextSplitter — The Sensible Default

RecursiveCharacterTextSplitter is the workhorse of the LangChain splitting ecosystem. It is the default choice in this book's pipeline and the right starting point for any new RAG project. Its central insight is simple: before falling back to arbitrary character positions, try to split on natural language boundaries — first paragraphs, then lines, then words, then characters — and only descend to a finer level when the coarser one fails to produce chunks small enough.

The separator cascade

The splitter accepts an ordered list of separators. At each step it tries the first separator; if the resulting pieces are still larger than chunk_size, it recurses down to the next separator. The default separator list used in this pipeline is:

```
separators=["\n\n", "\n", " ", ""]
```

This order encodes a priority: prefer to split on paragraph breaks (\n\n), then on line breaks (\n), then on word boundaries (space), and only as a last resort on individual characters (the empty string). The result is chunks that, wherever possible, contain complete paragraphs or at least complete sentences — far more semantically coherent than fixed-size cuts.

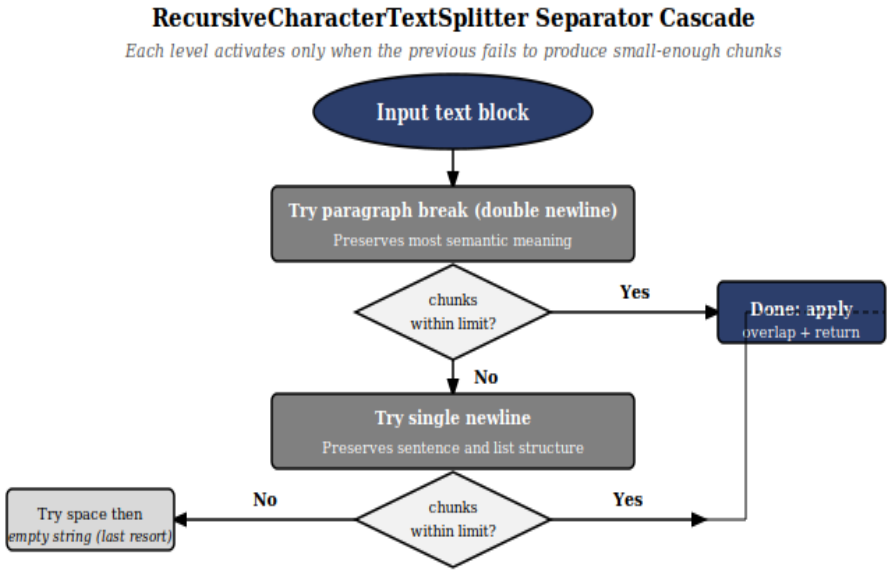

Figure 7.2 — The RecursiveCharacterTextSplitter separator cascade. The splitter descends one level only when the current level fails to produce chunks within chunk_size. Paragraph splits are always preferred over line splits, which are always preferred over word splits.

The overlap mechanism

After splitting, adjacent chunks share chunk_overlap characters at their boundary. With the default settings of chunk_size=1000 and chunk_overlap=200, the last 200 characters of chunk N become the first 200 characters of chunk N+1. This is not redundancy — it is insurance. A concept that straddles a split boundary is not lost entirely; it appears in full in at least one of the two neighboring chunks. Without overlap, boundary concepts would be permanently truncated from both neighbors.

Definition — RecursiveCharacterTextSplitter

A LangChain text splitter that divides documents by iterating through a priority-ordered list of separator strings. At each level it tries to split the text using the current separator; if the resulting pieces still exceed chunk_size, it recurses to the next separator in the list. The final step — splitting on the empty string — guarantees that no chunk exceeds chunk_size regardless of input structure. After splitting, a configurable chunk_overlap is applied, causing adjacent chunks to share characters at their boundary.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200,
    length_function=len,
    separators=["\n\n", "\n", " ", ""],
)

chunks = splitter.split_documents(documents)
print(f"Produced {len(chunks)} chunks from {len(documents)} documents")
```

7.4 Choosing chunk_size and chunk_overlap (1000 / 200 and When to Deviate)

The defaults used in this book's pipeline — chunk_size=1000 and chunk_overlap=200 — are not arbitrary. They represent a specific set of bets about the documents and queries the system will encounter. Understanding those bets is the prerequisite for knowing when to deviate from them.

Why 1000 characters?

One thousand characters is approximately one dense paragraph of English prose, or two shorter paragraphs. At this size, a typical chunk contains a complete thought — one argument, one process description, one factual claim — without merging unrelated topics from adjacent sections. The resulting embedding captures that thought with high fidelity.

At the token level (Section 7.5), 1000 characters is roughly 250 tokens of English text. With a top-k retrieval of 5 chunks, you consume approximately 1,250 tokens of context per retrieval call — a small fraction of the 128,000-token context window and well within the attention sweet spot.

Why 200 characters of overlap?

Two hundred characters is roughly 50 tokens — enough to capture a sentence or two that crosses a chunk boundary. The overlap is asymmetric in effect: it costs storage and compute at embedding time (those 200 characters appear in two consecutive embeddings), but it recovers information that would otherwise be lost. The 20% overlap ratio (200/1000) is a common convention; values between 10% and 25% are typical in production systems.

Scenario	Recommended chunk_size	Recommended overlap	Rationale
Short Q&A docs, FAQs	300–500 chars	50–100 chars	Each question-answer pair is a complete chunk; small overlap sufficient
Dense technical prose	800–1200 chars	150–250 chars	Default range; one concept per chunk, overlap protects boundaries
Legal contracts, policies	1500–2000 chars	300–400 chars	Clauses and sub-clauses span multiple paragraphs; larger windows needed
Source code files	500–800 chars	100–200 chars	Functions and methods are the logical unit; use code-aware splitter (§7.7)
Tables and structured data	50–200 chars per row	0 chars	Rows are independent; overlap introduces noise across row boundaries

Analogy — The sliding window on a newspaper roll

Imagine reading a long newspaper article through a sliding window that shows only 1000 characters at a time, advancing 800 characters per step (since 200 characters of overlap means the window rewinds by 200 before each step). Each window position is one chunk. The overlap ensures that any sentence spanning a window boundary appears complete in at least one of the two adjacent windows — the reader never sees half a sentence.

7.5 Chars vs Tokens — Why "1000 chars ≈ 250 tokens" and When the Ratio Breaks

The character-to-token ratio is not a constant. It is an approximation that holds well for plain English prose and breaks badly for code, URLs, mathematical notation, non-Latin scripts, and numeric data. Understanding when the ratio breaks matters for two reasons: it affects the actual token budget consumed by retrieved chunks, and it can cause silent context-window overflow when chunks are larger in tokens than the character count suggests.

Why the 4:1 ratio holds for English prose

Modern tokenizers — GPT's cl100k_base, LLaMA's SentencePiece, and most others — produce approximately one token per 4 characters for common English words. Common words like "the", "and", "is", "of" each cost one token. Slightly less common words like "retrieval" or "pipeline" may cost 1–2 tokens. The average across a typical English paragraph lands near 3.8–4.2 characters per token.

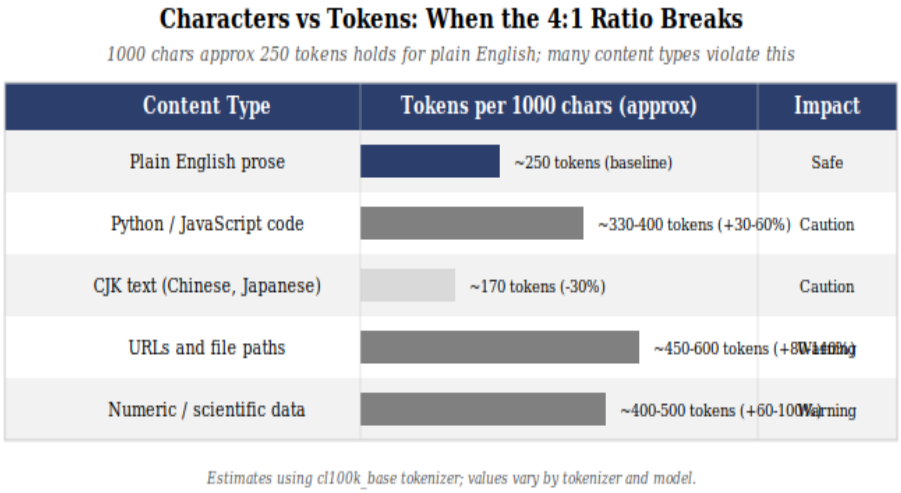

Figure 7.3 — The 4:1 character-to-token ratio holds for plain English prose but breaks for code, URLs, and non-Latin scripts. When chunking documents that contain these types, verify token counts directly rather than relying on character counts as a proxy.

When the ratio breaks

Code is token-expensive because identifiers, operators, punctuation, and whitespace are tokenized differently from prose. A line like `retriever.retrieve(query, top_k=5)` contains 34 characters but costs roughly 12–15 tokens — a 2.3:1 ratio rather than 4:1.

URLs and file paths are extreme cases. A URL like `https://api.example.com/v2/documents?filter=type:policy&limit=50` contains 65 characters but can cost 30+ tokens as the tokenizer fragments the path components and query string character-by-character.

CJK text (Chinese, Japanese, Korean) goes the other direction: each character is often a single token, so 1000 Chinese characters may cost only 170–200 tokens — well below the 250-token estimate.

Token-accurate chunking with tiktoken

For documents where the character-to-token ratio is unreliable, replace the default length_function=len with a token-counting function:

```
import tiktoken

enc = tiktoken.get_encoding("cl100k_base")  # GPT-4 tokenizer

def token_len(text: str) -> int:
    return len(enc.encode(text))

splitter = RecursiveCharacterTextSplitter(
    chunk_size=250,         # now in tokens, not chars
    chunk_overlap=50,       # 50-token overlap
    length_function=token_len,
    separators=["\n\n", "\n", " ", ""],
)
```

Note that when you switch to token-counting, the chunk_size and chunk_overlap values change units: 250 tokens is your target chunk size, not 250 characters. This is generally the right approach for mixed-content corpora containing both prose and code.

Common pitfall — Measuring chunk tokens only at ingest time

The token count you observe when splitting does not include the metadata prefix that the retrieval pipeline prepends to each chunk before sending it to the LLM. A standard source citation — "[Source: contract_v3.pdf, page 7]" — adds 10–15 tokens per chunk. With top-k=5 retrievals, that is 50–75 extra tokens of overhead. For a pipeline running near a token budget limit, this overhead can push individual retrievals past the threshold. Measure the full formatted chunk, not just the page_content field.

7.6 Semantic Chunking — Splitting by Meaning, Not Characters

Recursive splitting respects linguistic structure (paragraphs, sentences) but is blind to semantic structure — it cannot tell whether two adjacent paragraphs are discussing the same concept or have shifted to a completely different topic. Semantic chunking addresses this by measuring the embedding similarity between consecutive sentences and inserting a split wherever the similarity drops sharply — at the point of a topic transition.

Definition — Semantic Chunking

A splitting strategy that embeds each sentence (or small window of sentences) independently and detects topic-shift boundaries by measuring cosine similarity between adjacent sentence embeddings. A split is inserted wherever the similarity drops below a configurable threshold, producing chunks that each contain a single coherent topic. Semantic chunking produces more precise chunks than recursive splitting but requires an embedding call for every sentence at ingest time, making it significantly slower.

How it works

The algorithm operates in three phases. First, every sentence in the document is embedded individually. Second, the similarity between each consecutive pair of sentence embeddings is computed; a sharp drop — typically below a threshold set by percentile (e.g., the bottom 5th percentile of similarities in this document) — is flagged as a boundary. Third, sentences on the same side of a boundary are merged into a single chunk.

```
from langchain_experimental.text_splitter import SemanticChunker
from langchain_huggingface import HuggingFaceEmbeddings

embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)

splitter = SemanticChunker(
    embeddings=embeddings,
    breakpoint_threshold_type="percentile",  # or 'standard_deviation'
    breakpoint_threshold_amount=95,           # split at lowest 5%
similarities
)

chunks = splitter.split_documents(documents)
```

When to use semantic chunking

Semantic chunking produces demonstrably better retrieval precision on documents with clear topic structure — academic papers, technical reports, structured manuals. Each chunk is a genuine topic unit rather than a character-count window. The tradeoff is cost: embedding every sentence during ingestion adds significant time, and the chunk sizes are variable and unpredictable (some topic sections may be one sentence; others may span twenty).

For the base pipeline in this book, RecursiveCharacterTextSplitter remains the default because its performance-to-cost ratio is superior for general use. Semantic chunking is the recommended upgrade path when retrieval quality diagnostics (Section 7.10) reveal that the recursive splitter is cutting across topic boundaries in your specific corpus.

> **Analogy — Chapter boundaries vs page boundaries**
>
> Fixed-size and recursive splitters cut at page boundaries — after a fixed amount of content regardless of where the author's thought happens to end. Semantic chunking finds chapter boundaries — the natural points where one idea stops and another begins. Chapter boundaries make better retrieval units because a reader asking about a specific topic expects the answer to come from one chapter, not from the last paragraph of one chapter and the first paragraph of the next.

7.7 Markdown-Aware and Code-Aware Splitters

Many document corpora contain structured markup — Markdown wikis, README files, Jupyter notebooks, and source code — where naive character or paragraph splitting discards structural information that is directly useful for retrieval. LangChain provides two specialist splitters that understand and exploit these structures.

MarkdownTextSplitter and MarkdownHeaderTextSplitter

MarkdownTextSplitter uses Markdown-specific separators — heading markers (##), horizontal rules (---), fenced code blocks (```), and then standard whitespace — in the separator cascade. It keeps fenced code blocks intact rather than cutting through them.

MarkdownHeaderTextSplitter goes further: it identifies each heading level (H1, H2, H3) and promotes them to metadata fields on each chunk. A chunk produced from a section under a "## Installation" heading will carry {"Header 2": "Installation"} in its metadata, enabling precise metadata-filtered retrieval later.

```
from langchain_text_splitters import MarkdownHeaderTextSplitter

headers_to_split_on = [
    ("#",  "h1"),
    ("##", "h2"),
    ("###","h3"),
]

splitter = MarkdownHeaderTextSplitter(
    headers_to_split_on=headers_to_split_on,
    strip_headers=False,  # keep headers in chunk text
)

chunks = splitter.split_text(markdown_text)
# Each chunk.metadata now contains h1, h2, h3 fields
```

RecursiveCharacterTextSplitter with language mode

For source code, RecursiveCharacterTextSplitter supports a from_language factory that loads language-specific separator lists. Python separators, for example, prioritise class and function definitions (\nclass , \ndef) before falling back to indentation and whitespace. This keeps function bodies intact and avoids splits in the middle of a method signature or docstring.

```
from langchain_text_splitters import Language,
RecursiveCharacterTextSplitter

python_splitter = RecursiveCharacterTextSplitter.from_language(
    language=Language.PYTHON,
    chunk_size=800,
    chunk_overlap=150,
)

# Other supported languages: JS, TS, GO, RUST, JAVA, C, CPP, MARKDOWN,
HTML
js_splitter = RecursiveCharacterTextSplitter.from_language(
    language=Language.JS,
    chunk_size=600,
    chunk_overlap=100,
)
```

> **Common pitfall — Using the generic splitter on source code**
>
> The default separator list ["\n\n", "\n", " ", ""] does not know that \ndef marks the start of a Python function or that \nfunc marks the start of a Go function. A generic splitter will happily cut through a function definition, placing the signature in one chunk and the body in another. The resulting chunks are syntactically broken — neither can be understood in isolation. Always use from_language() or a language-specific separator list when chunking code.

7.8 Document-Structure-Aware Chunking (Headers, Sections)

Some documents have explicit structural metadata that is more informative than any heuristic boundary detection. Academic papers have abstracts, introduction, methods, results, and discussion sections. Regulatory documents have numbered clauses and sub-clauses. Technical manuals have chapters, appendices, and glossaries. When this structure is available — either from the file format itself (HTML section tags, DOCX heading styles, PDF bookmarks) or from consistent heading patterns — splitting along those structural boundaries produces chunks that are semantically and administratively coherent.

HTMLHeaderTextSplitter

For HTML documents, HTMLHeaderTextSplitter identifies heading tags (h1 through h4) and uses them as split points, propagating the heading text as metadata fields on each resulting chunk. This is the counterpart of MarkdownHeaderTextSplitter for web-sourced content.

```
from langchain_text_splitters import HTMLHeaderTextSplitter

headers_to_split_on = [
    ("h1", "section"),
    ("h2", "subsection"),
    ("h3", "subsubsection"),
]

splitter =
HTMLHeaderTextSplitter(headers_to_split_on=headers_to_split_on)
chunks = splitter.split_text(html_string)

# chunks[0].metadata = {
#   'section':    'Introduction',
#   'subsection': 'Background',
# }
```

Building a header-aware pipeline for PDFs

PDFs do not natively expose heading structure, but font size and weight information from PyMuPDFLoader can be used as a proxy. A common approach is to run a first-pass scan of the PDF to identify paragraphs rendered at larger font sizes as likely section headings, then split on those boundaries:

```
import fitz  # PyMuPDF

def extract_sections_by_font(pdf_path: str) -> list[dict]:
    """Heuristically identify section boundaries using font size."""
    doc = fitz.open(pdf_path)
    sections = []
    current_section = {'heading': '', 'content': ''}

    for page in doc:
        for block in page.get_text('dict')['blocks']:
            for line in block.get('lines', []):
                for span in line.get('spans', []):
                    if span['size'] > 14:  # treat as heading
                        if current_section['content']:
                            sections.append(current_section)
                        current_section = {
                            'heading': span['text'],
                            'content': '',
                        }
                    else:
                        current_section['content'] += span['text'] + ' '

    if current_section['content']:
        sections.append(current_section)
    return sections
```

The advantage of document-structure-aware chunking is that metadata-filtered retrieval becomes highly precise: a query for "methods section" or "clause 4.2" can be routed directly using metadata filters rather than relying on semantic similarity alone. The disadvantage is the extra parsing complexity and the fragility of font-size heuristics on poorly formatted PDFs.

7.9 Chunk Hygiene — Stripping Boilerplate, Deduplication, Length Filters

Splitting produces raw chunks. Raw chunks are not clean chunks. Before embedding, a short sequence of quality filters — collectively called chunk hygiene — should be applied to remove stub chunks, strip irrelevant boilerplate, and deduplicate near-identical content. This section covers the three most important hygiene steps and the code patterns used to implement them in this book's pipeline.

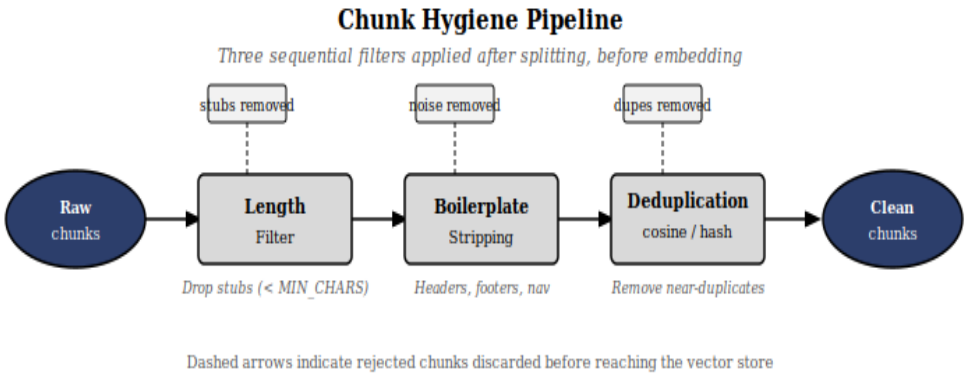

Figure 7.5 — The chunk hygiene pipeline. Three sequential filters applied after splitting and before embedding remove stubs, boilerplate, and near-duplicates. Only clean chunks reach the vector store.

Length filtering

Splitting frequently produces stub chunks — fragments shorter than any useful semantic unit. These arise from document headers, page numbers, footnote markers, table-of-contents entries, and section-break artifacts. Stubs consume vector DB slots, dilute retrieval rankings with irrelevant matches, and produce noisy citations.

The pipeline in this book applies a minimum chunk length filter immediately after splitting:

```
# ingest.py — the MIN_CHUNK_CHARS guard (already in the codebase)
MIN_CHUNK_CHARS = 50   # configurable constant

before = len(chunks)
chunks = [c for c in chunks
          if len(c.page_content.strip()) >= MIN_CHUNK_CHARS]
print(f"Filtered {before - len(chunks)} stub chunks. {len(chunks)}
remain.")
```

Fifty characters is a conservative minimum — roughly half a sentence. You may raise this for corpora with dense prose (100–150 characters is common) or lower it for datasets where short factual statements ("Capital of France: Paris") are legitimate retrieval targets.

Boilerplate stripping

Long documents frequently contain repeated boilerplate: running headers and footers, copyright notices, navigation menus, table-of-contents entries, and legal disclaimers. These fragments appear in many chunks and match a wide range of queries without contributing useful information — they are retrieval noise.

```
import re

BOILERPLATE_PATTERNS = [
    r'Page \d+ of \d+',           # page numbers
    r'Copyright \d{4}.*?\.\s',   # copyright lines
    r'All rights reserved\.?\s',
    r'Table of Contents',
    r'\bConfidential\b',
]

def strip_boilerplate(text: str) -> str:
    for pattern in BOILERPLATE_PATTERNS:
        text = re.sub(pattern, '', text, flags=re.IGNORECASE)
    return ' '.join(text.split())  # normalise whitespace

chunks = [
    type(c)(page_content=strip_boilerplate(c.page_content),
            metadata=c.metadata)
    for c in chunks
]
```

Deduplication

When the same document is ingested twice — either accidentally or because a parent directory and a child directory both appear in the DATA_DIRS list — near-identical chunks appear in the vector store. They consume slots, inflate the apparent corpus size, and cause the retriever to return near-duplicate results that waste context-window tokens.

Two deduplication strategies are common: exact hash deduplication (fast, catches only exact duplicates) and cosine-similarity deduplication (slower, catches near-duplicates including minor reformatting). The pipeline in this book uses the ingest_run_id metadata field for exact deduplication on re-ingestion, and cosine similarity at query time for near-duplicate compression (see Chapter 22B).

```
import hashlib

def dedup_exact(chunks: list) -> list:
    """Remove exact duplicate chunks by content hash."""
    seen = set()
    unique = []
    for chunk in chunks:
        h = hashlib.md5(chunk.page_content.encode()).hexdigest()
        if h not in seen:
            seen.add(h)
            unique.append(chunk)
    removed = len(chunks) - len(unique)
    if removed:
        print(f"Dedup removed {removed} exact duplicate chunks")
    return unique
```

Common pitfall — Deduplicating after embedding instead of before

Computing a cosine similarity matrix across all chunks requires $O(n^2)$ embedding comparisons. On a corpus of 10,000 chunks this is 100 million comparisons — computationally expensive and memory-intensive. Exact hash deduplication before embedding is free and catches the most common case (accidental double-ingestion). Apply it first; apply semantic deduplication only if exact deduplication is insufficient for your corpus.

7.10 Experimenting: How Chunking Choices Visibly Change Retrieval Quality

Chunking strategy is not a set-and-forget decision. The right configuration depends on your specific corpus, your typical query patterns, and the downstream LLM's context window. The most reliable way to choose is to run a controlled experiment: ingest the same corpus with different chunk configurations, issue a fixed query set, and measure the results.

Setting up a comparison experiment

The simplest meaningful comparison tests three configurations: a small-chunk setup (300 chars / 50 overlap), the default (1000 chars / 200 overlap), and a large-chunk setup (2000 chars / 400 overlap). For each configuration, ingest into a separate ChromaDB collection, retrieve the top-5 chunks for each query, and score the results.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
from embedding_manager import EmbeddingManager
from vector_store import VectorStore

CONFIGS = [
    {"name": "small",   "size": 300,  "overlap": 50},
    {"name": "default", "size": 1000, "overlap": 200},
    {"name": "large",   "size": 2000, "overlap": 400},
]

TEST_QUERIES = [
    "What is the minimum chunk size recommended?",
    "How does overlap prevent information loss at boundaries?",
    "When should I use semantic chunking instead of recursive?",
]
embedding_manager = EmbeddingManager()

for cfg in CONFIGS:
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=cfg["size"],
        chunk_overlap=cfg["overlap"],
        length_function=len,
        separators=["\n\n", "\n", " ", ""],
    )
    chunks = splitter.split_documents(documents)
    store = VectorStore(collection_name=f"exp_{cfg['name']}")
    store.add_documents_in_batches(chunks, embedding_manager)
    print(f"{cfg['name']}: {len(chunks)} chunks")
```

What to look for in the results

The experiment will typically reveal a consistent pattern across different corpora. Small chunks (300 chars) retrieve with high precision for narrow factual queries — single-sentence questions that have a one-sentence answer in the document. For multi-sentence questions or questions requiring context across a paragraph, small chunks miss the surrounding explanation and return brittle, context-free fragments.

Large chunks (2000 chars) rarely miss the relevant passage, but they bring substantial irrelevant material as neighbors. The LLM must search within each chunk to find the relevant sentence — a task it does reliably at 2000 tokens but far less reliably as chunks grow larger. The context window fills faster, limiting how many diverse topic areas a single query can cover.

The default (1000 chars) performs well across both query types. It is not optimal for either extreme, but it degrades gracefully. This is exactly the property you want from a default: not the best choice for any specific scenario, but the least-bad choice across all scenarios.

Config	Chunk count (same corpus)	Precision (narrow queries)	Recall (broad queries)	Context cost (top-5 retrieval)
Small (300/50)	~3× more chunks	★★★★★ Excellent	★★★ Moderate	~375 tokens
Default (1000/200)	Baseline	★★★★ Good	★★★★ Good	~1,250 tokens
Large (2000/400)	~2× fewer chunks	★★★ Moderate	★★★★★ Excellent	~2,500 tokens

Automating the quality measurement

Manual inspection of retrieved chunks is feasible for small query sets but does not scale. A more systematic approach uses a relevance judge — the same LLM-as-judge pattern used in Chapter 21 — to rate each retrieved chunk as RELEVANT, PARTIAL, or IRRELEVANT for the given query. Aggregate these ratings across the query set and you have a quantitative retrieval quality score for each chunk configuration:

```
def score_config(collection_name: str, queries: list[str],
                 embedding_manager, llm) -> float:
    store = VectorStore(collection_name=collection_name)
    retriever = RAGRetriever(store, embedding_manager)
    total, relevant = 0, 0

    for query in queries:
        chunks = retriever.retrieve(query, top_k=5)
        for chunk in chunks:
            verdict = relevance_judge(query, chunk['content'], llm)
            if verdict == 'RELEVANT':  relevant += 1
            if verdict != 'SKIPPED':   total += 1

    return relevant / total if total else 0.0

for cfg in CONFIGS:
    score = score_config(f"exp_{cfg['name']}", TEST_QUERIES,
                         embedding_manager, llm)
    print(f"{cfg['name']:10s}: {score:.1%} relevance")
```

Definition — Retrieval Precision

The fraction of retrieved chunks that are judged RELEVANT or PARTIAL to the query that triggered their retrieval. A retrieval precision of 0.80 means that 4 out of every 5 returned chunks contribute useful information to the answer. Retrieval precision is the key diagnostic metric for chunking experiments: it captures whether your chunk size is producing focused, answerable units, without conflating failures attributable to embedding quality or retriever ranking.

The practical advice is simple: run this experiment on your actual corpus with your actual queries before committing to a chunk configuration. Thirty minutes of offline experimentation can save weeks of debugging poor retrieval quality in production. The numbers above (Table 7.2) are patterns, not guarantees — your corpus may have characteristics that invert them entirely.

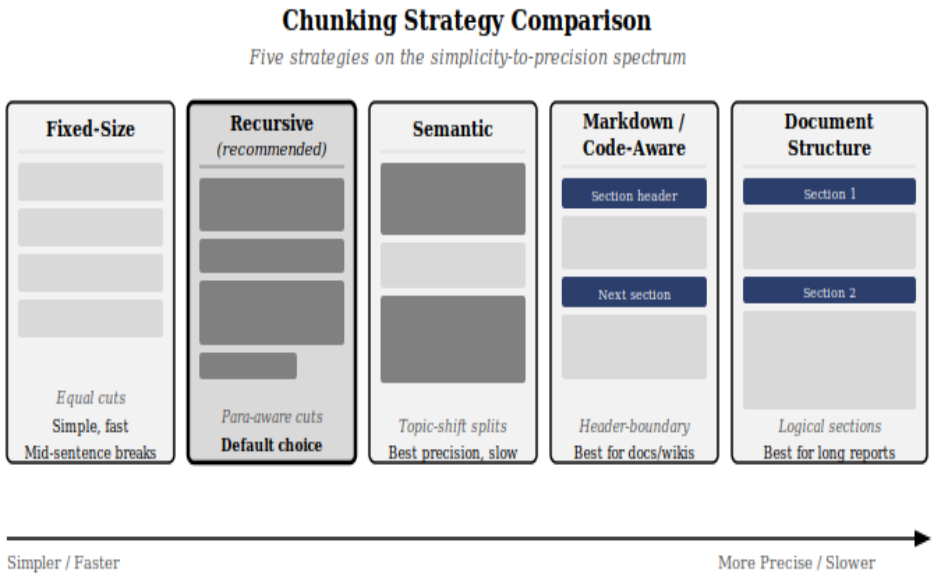

Figure 7.4 — Five chunking strategies on the precision–context spectrum. Recursive splitting (★) is the recommended default because it degrades gracefully across document types. Semantic chunking delivers superior precision at the cost of ingestion time. Choose based on your corpus and retrieval quality diagnostics.

Chunking is the last purely text-manipulation step in the ingestion pipeline. Everything after it operates on vectors. Chapter 8 — Embeddings Deep Dive — examines how those chunks are converted into dense numerical representations, why the model choice matters, and what it means for a 384-dimensional vector to carry the semantic content of your carefully produced chunk.